

Transformers

Hrachya Asatryan

The problem with Seq2Seq

- **The Sequential Trap:** In RNNs and LSTMs, generating the hidden state h_t strictly requires the previous state h_{t-1} .
- **$O(N)$ Sequential Operations:** If a sequence has 1,000 tokens, the network must perform 1,000 sequential steps. You cannot compute token 10 until token 9 is finished.
- **The Hardware Bottleneck:** Modern Deep Learning relies on GPUs, which excel at massive parallelization. Sequential models inherently underutilize GPU compute power because they cannot process the entire sequence at once.

The problem with Seq2Seq

A Simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax}(Uh^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

Core idea: Apply the same weights W repeatedly

hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

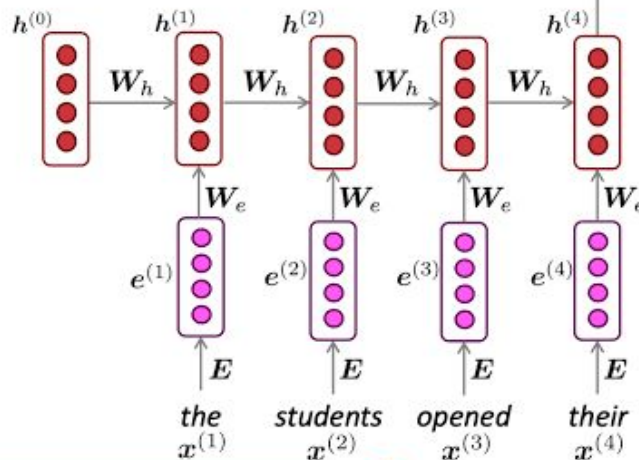
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = Ex^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$

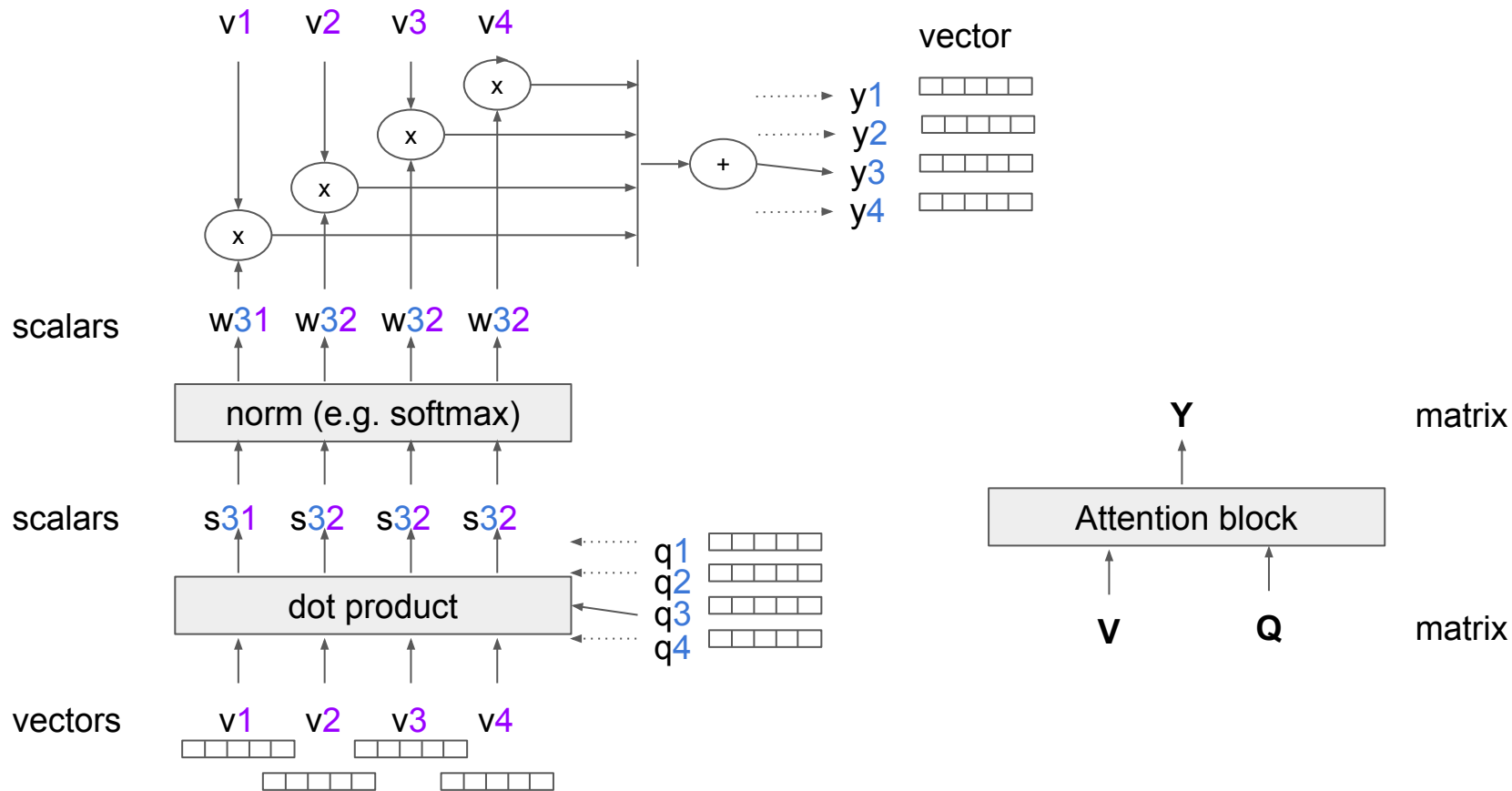


Note: this input sequence could be much longer now!

Attention Is All You Need

- Up until 2017, Attention was just a *helper* mechanism used to improve the performance of RNNs.
- **The Core Hypothesis:** What if we drop recurrence entirely?
- If we can build a model entirely out of Attention, we can process all tokens simultaneously.
- **The Benefit:** Massive parallelization across GPUs, leading to drastically reduced training times and the ability to train on vastly larger datasets.

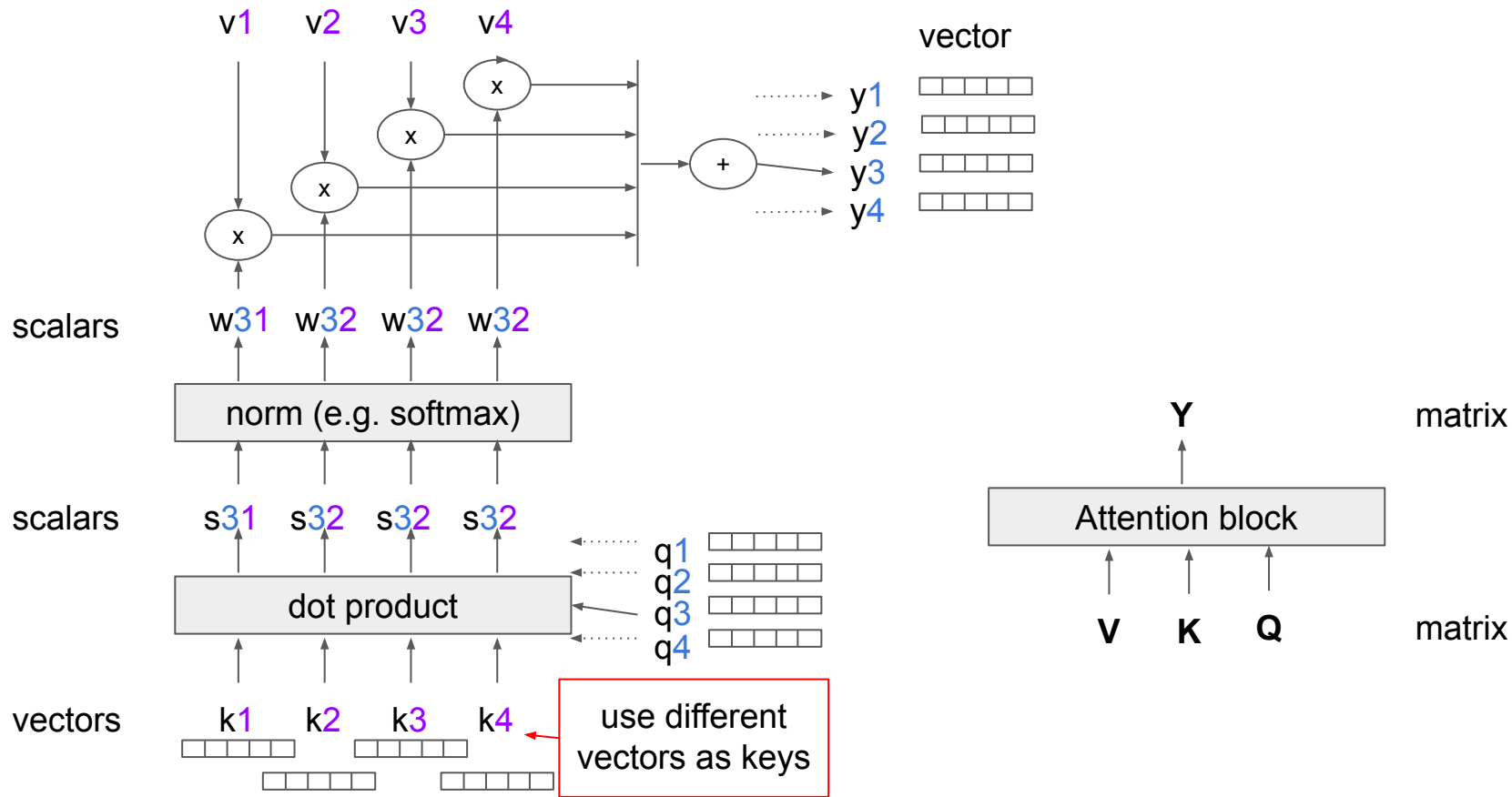
Attention



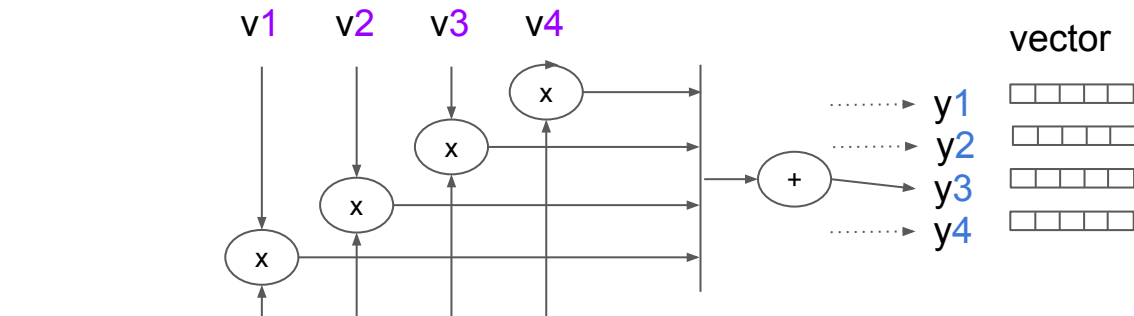
Attention

- More generally, given a set of (**key**⁽ⁱ⁾, **value**⁽ⁱ⁾) vector pairs, and a vector **query**, attention is a mechanism to compute a weighted sum of **values** dependent on attention scores between the **query** and the **keys**
- In our previous examples, **key = value**

Attention



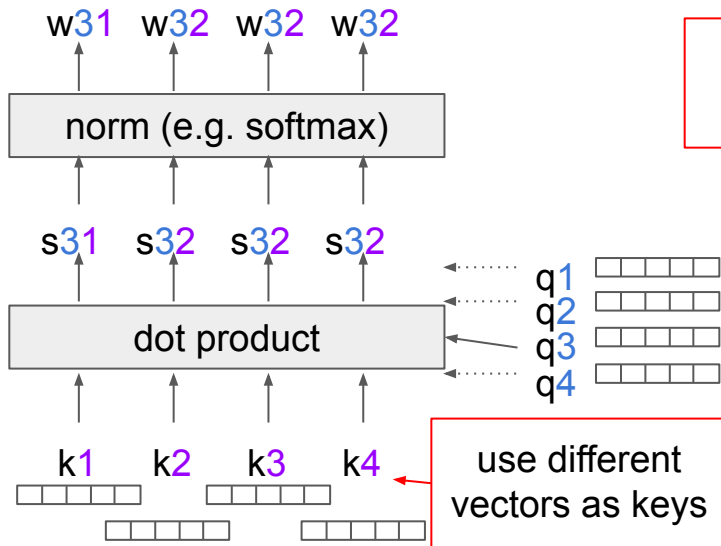
Attention



scalars

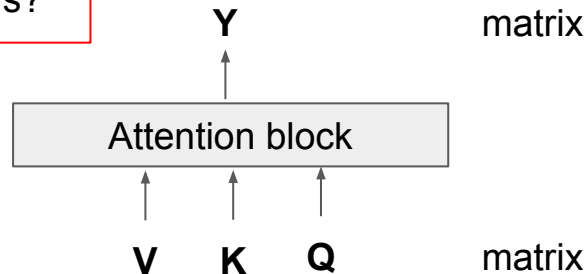
scalars

vectors

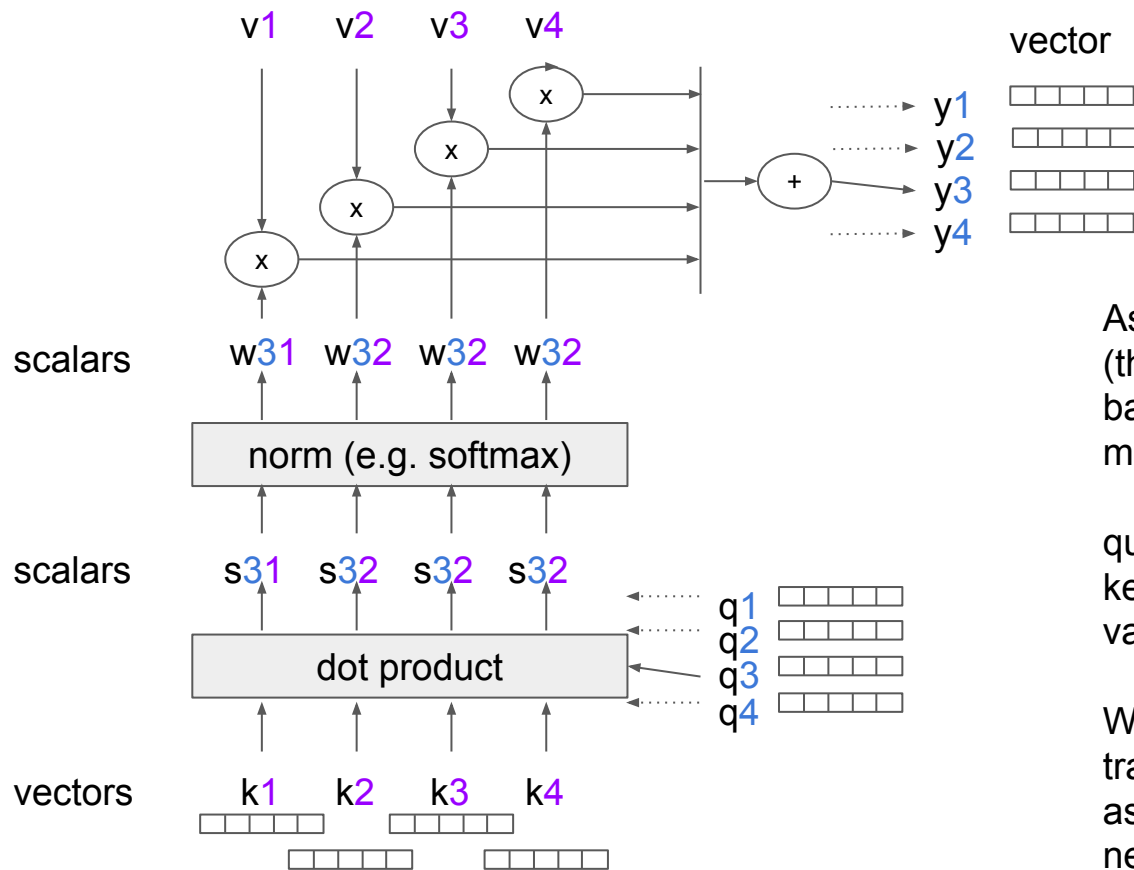


but what are
these keys?

use different
vectors as keys



Generalized Attention Block

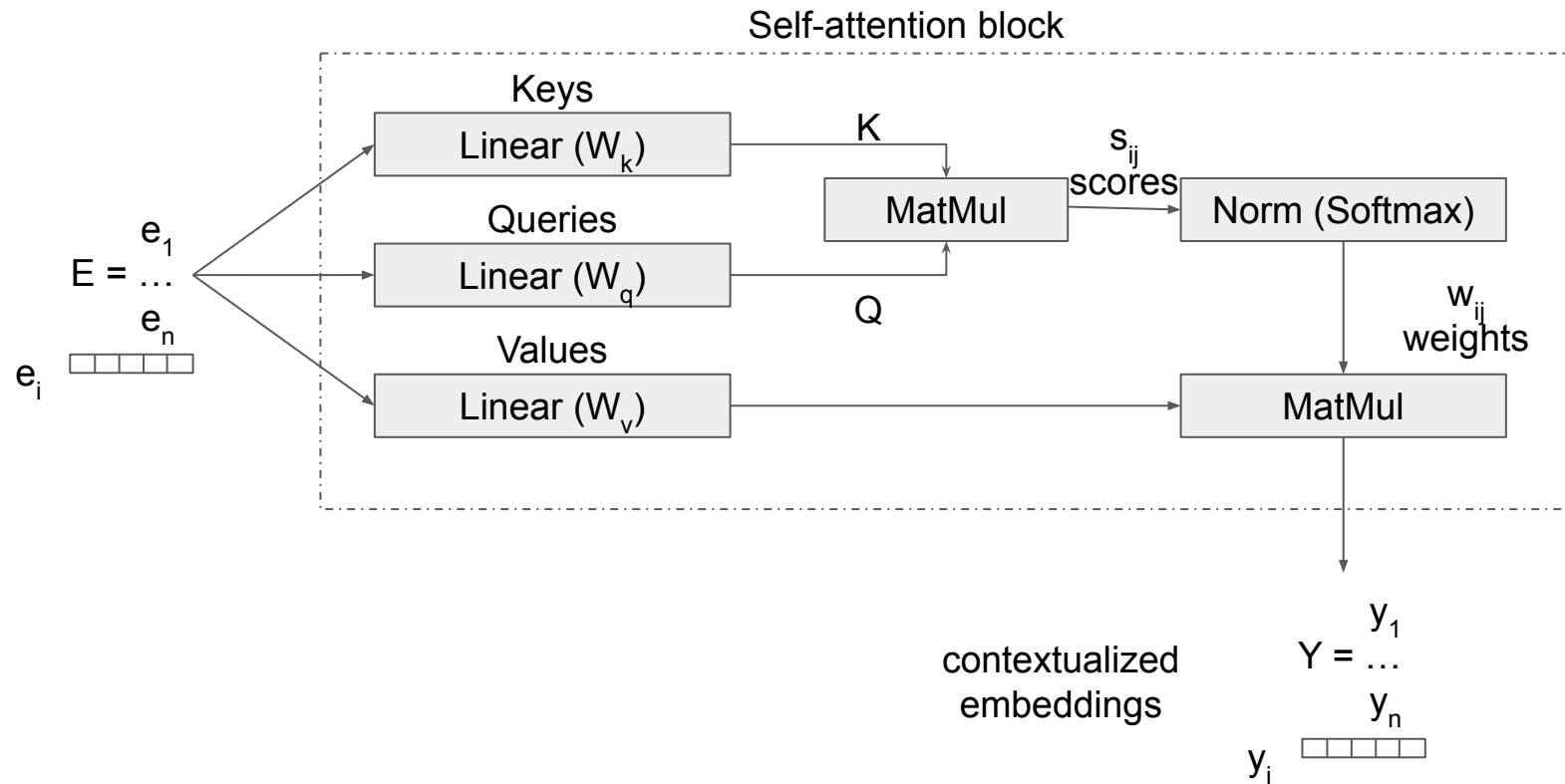


Assume some embedding matrix E
(these were word embeddings in our
basic example, or hidden states for
machine translation)

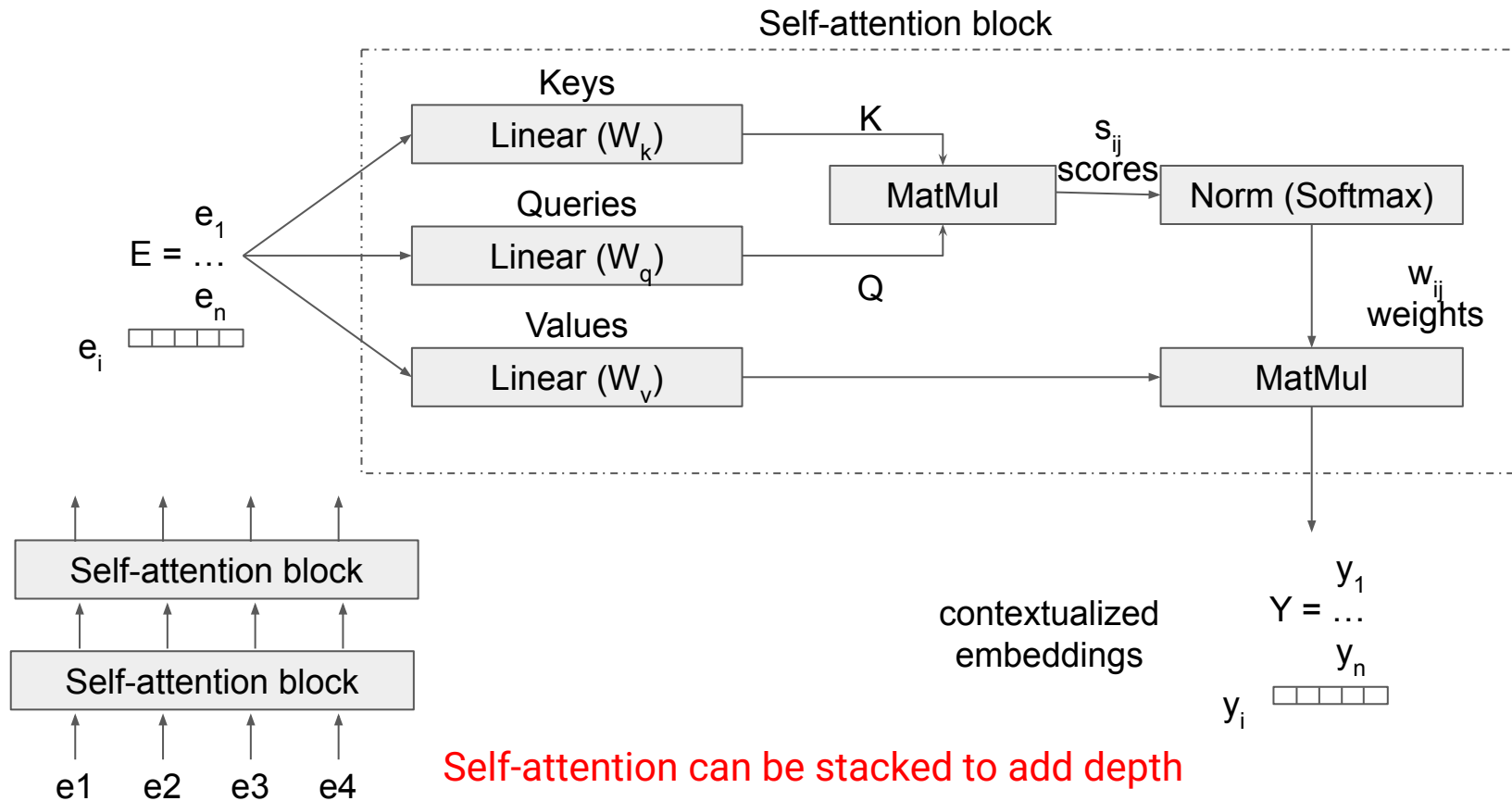
$$\begin{aligned} \text{query: } Q &= E \times W_q \\ \text{keys: } K &= E \times W_k \\ \text{value: } V &= E \times W_v \end{aligned}$$

Weights W perform a linear
transformation, and can be represented
as weights of a single-layer linear neural
network.

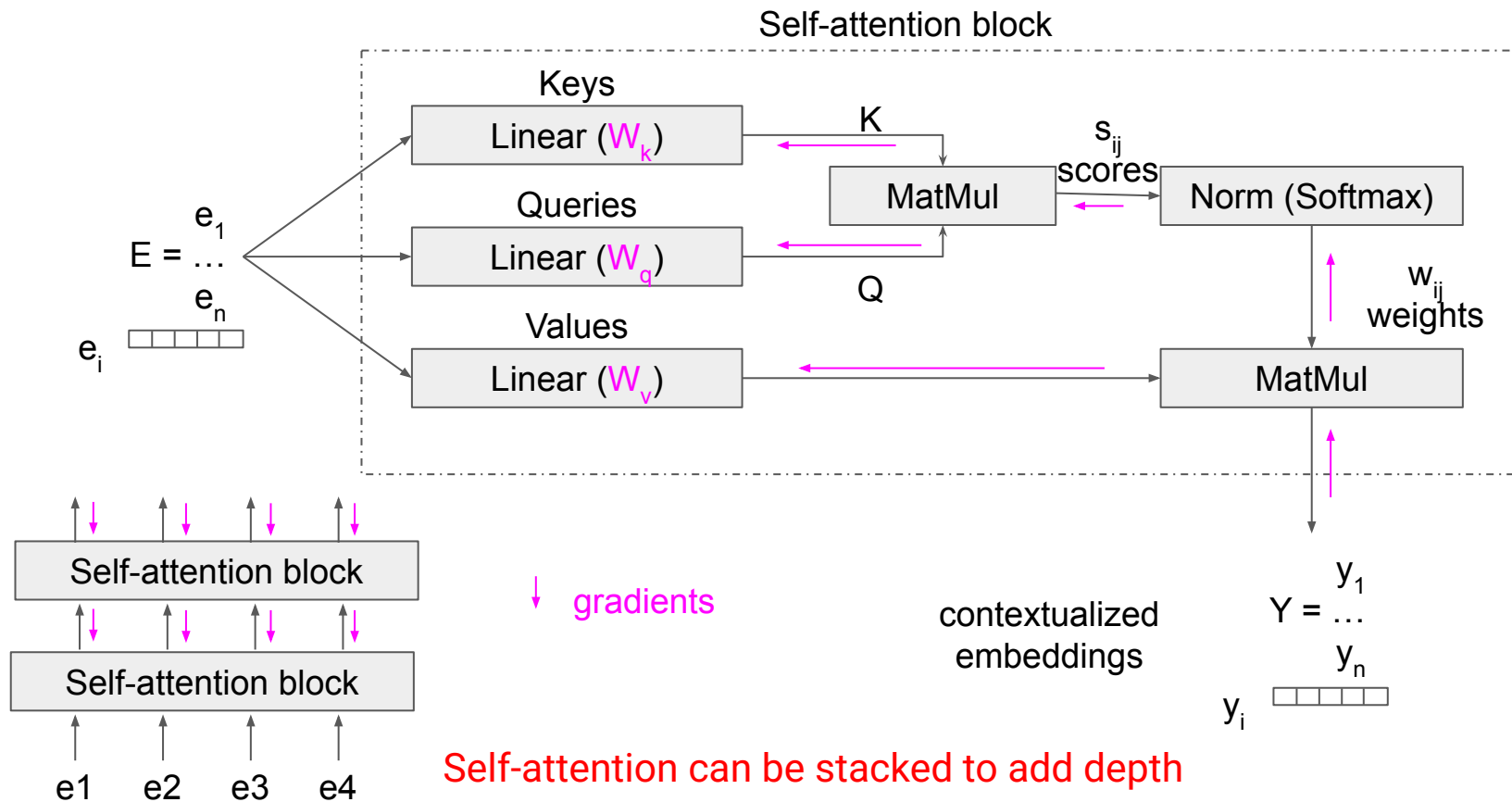
Generalized Attention Block



Generalized Attention Block



Generalized Attention Block



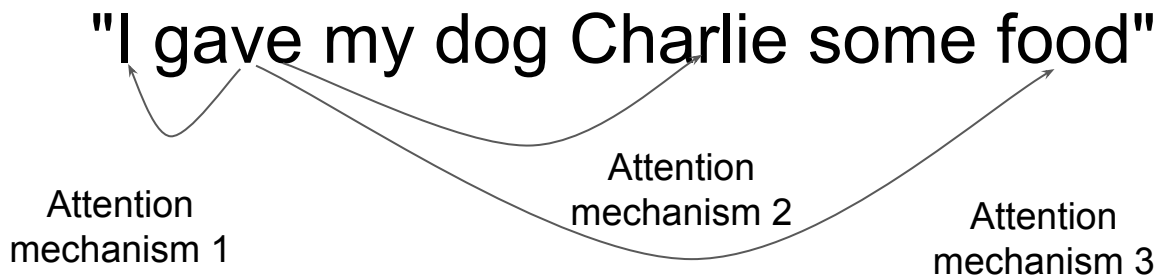
Multi-headed Attention

We might often want to pay attention to different parts of the sentence in different ways, to capture different types of relationships between words e.g.

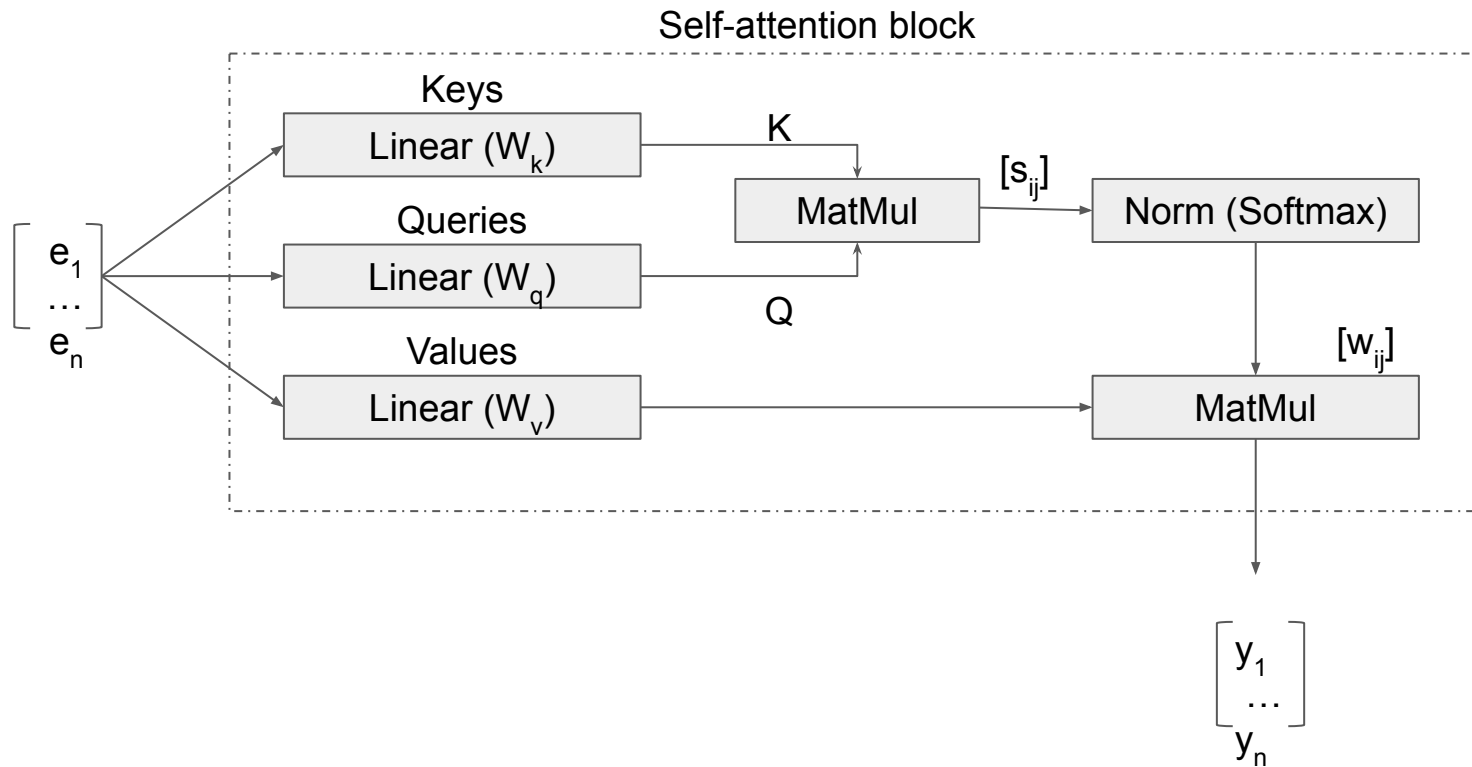


Multi-headed Attention

We might often want to pay attention to different parts of the sentence in different ways, to capture different types of relationships between words e.g.

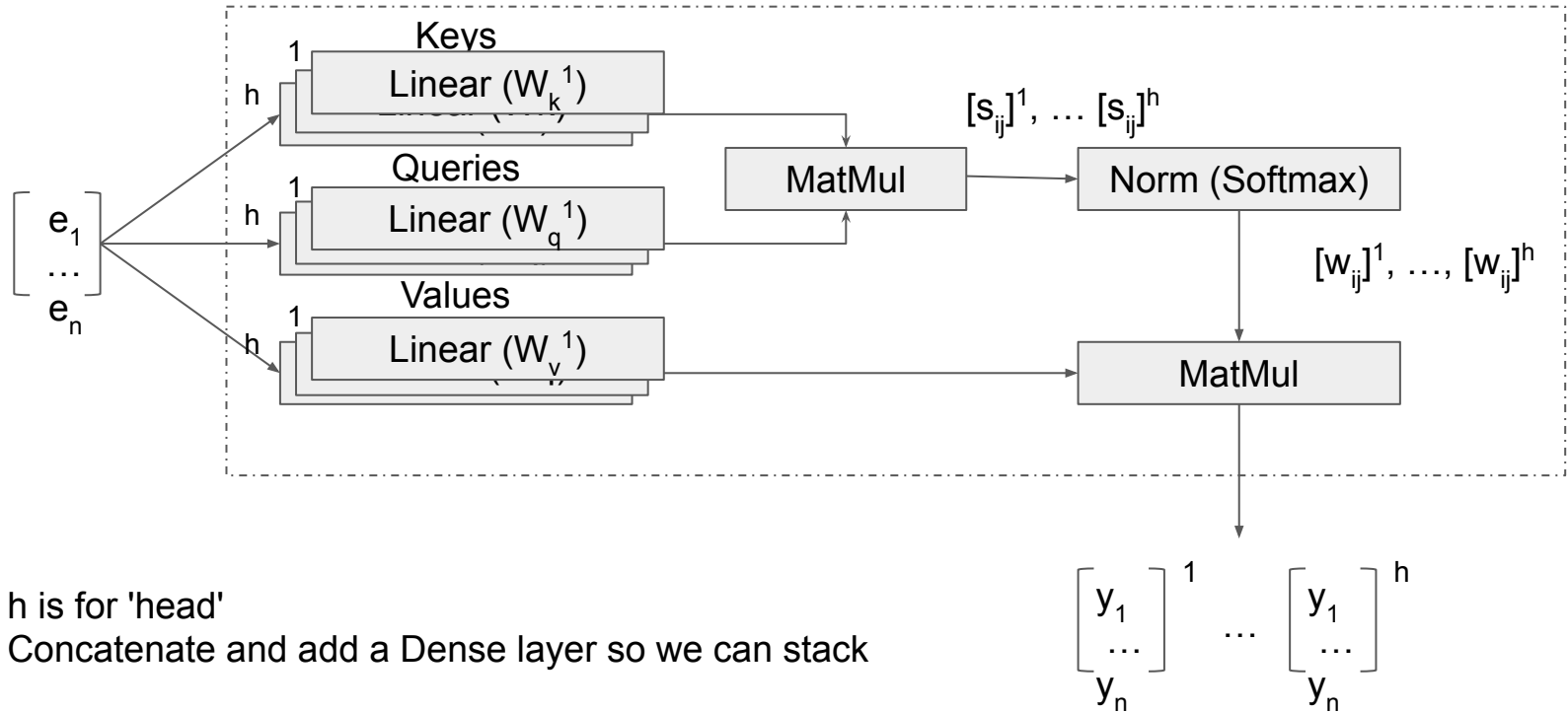


Generalized Attention Block



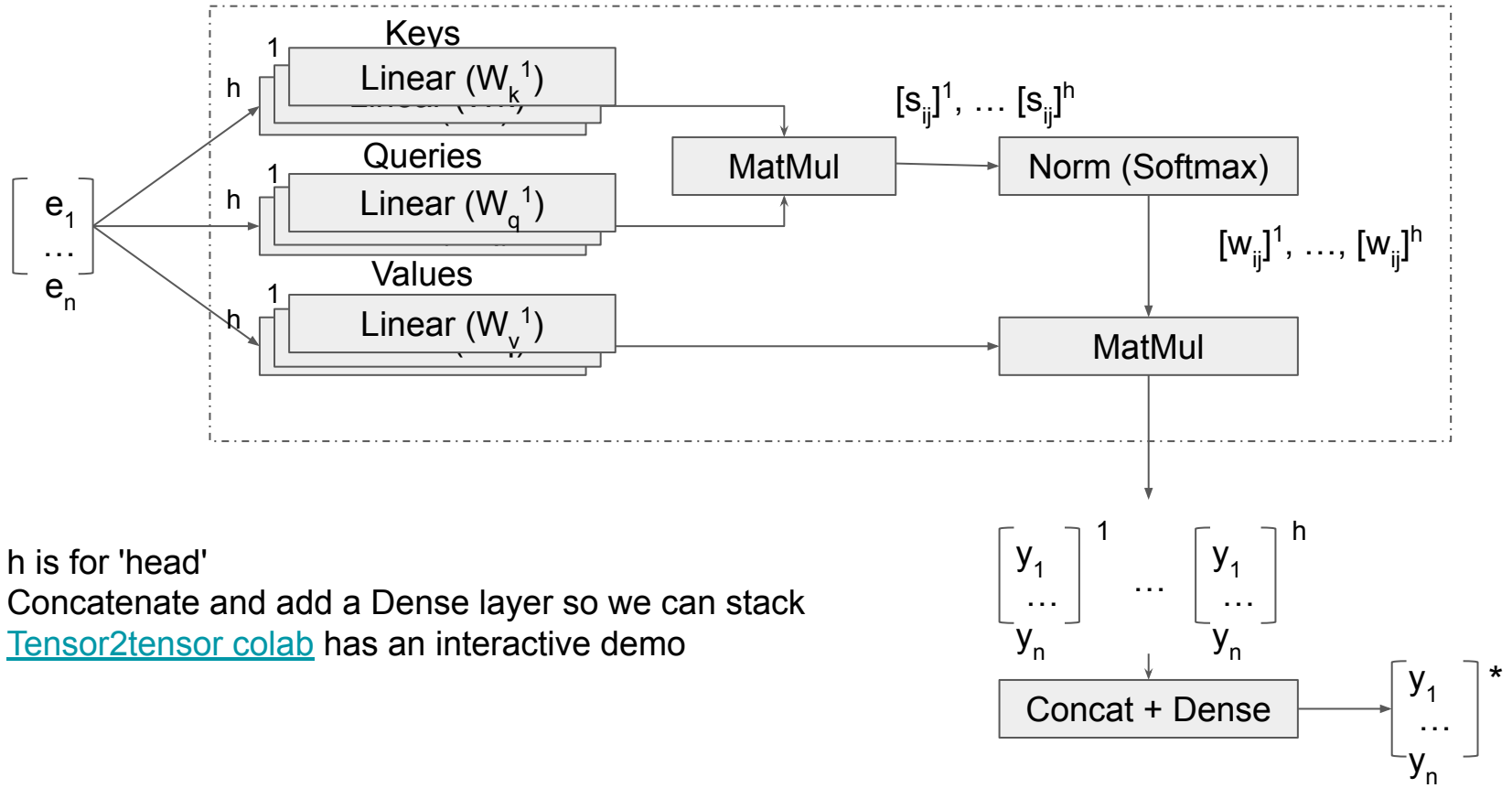
Multi-headed Attention

Self-attention block



Multi-headed Attention

Self-attention block

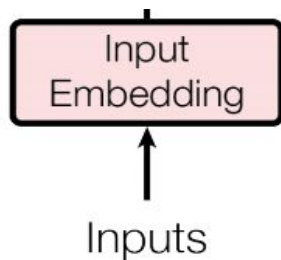


Sub-word Embeddings

- **The Extremes:** Word-level tokenization requires massive vocabularies and struggles with new words. Character-level tokenization creates sequences that are too long for the model to remember.
- **The Middle Ground (Byte-Pair Encoding):** Start with characters, find the most frequent adjacent pairs in the training data, and merge them iteratively.
- **Solving OOV:** This solves the Out-Of-Vocabulary (OOV) problem. If the model encounters a completely novel word, it simply breaks it down into known sub-word chunks.

Sub-word Embeddings

- The vocabulary is ~30,000 words and sub-words, including about 1000 single character tokens, many 2-character tokens, etc.
- Words are often split into subwords by the tokenizer.
- The most common token length is 6.
- The embedding dimension is 512



embeddings -> "emb" "bed" "ding"

- Treated as 3 separate words.

tokens at the beginning and in the middle of words are treated differently

- "emb" "##bed" "##ding"

Learned Embeddings

- Once text is tokenized, we cannot feed discrete IDs directly into a neural network.
- We map each token to a continuous vector space using a trainable Embedding Matrix $E \in \mathbb{R}^{|V| \times d}$, where $|V|$ is the vocabulary size and d is the embedding dimension (e.g., 512).
- Unlike static embeddings (like Word2Vec), these embeddings are learned and updated dynamically during the model's training process to suit the specific task.

Learned Embeddings

Word Vector for hope = $W_{3 \times 5} \times V_{5 \times 1}$

w00	w01	w02	w03	w04
w10	w11	w12	w13	w14
w20	w21	w22	w23	w24

 $\times$

1
0
0
0
0

 $=$

w00
w10
w20

$V_{3 \times 1}$

- Hidden layer acts as a look-up table - *Selects* the matrix row corresponding to the "1"

$$[0 \ 0 \ 0 \ 1 \ 0] \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = [10 \ 12 \ 19]$$

The "Bag of Words" Problem

- By removing recurrence to process everything in parallel, we created a new problem: the model is now **permutation invariant**.
- To a pure self-attention mechanism, the sequence of tokens is just an unordered "bag of words."
- The math treats "The dog bit the man" exactly the same as "The man bit the dog."
- We must explicitly inject a sense of time and sequence order back into our data before it reaches the attention block.

Positional Encoding

- Since the order of the words does not matter for the attention block, we add a vector to the embedding to encode the position of the word

$$\vec{p}_t^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k \cdot t), & \text{if } i = 2k \\ \cos(\omega_k \cdot t), & \text{if } i = 2k + 1 \end{cases}$$

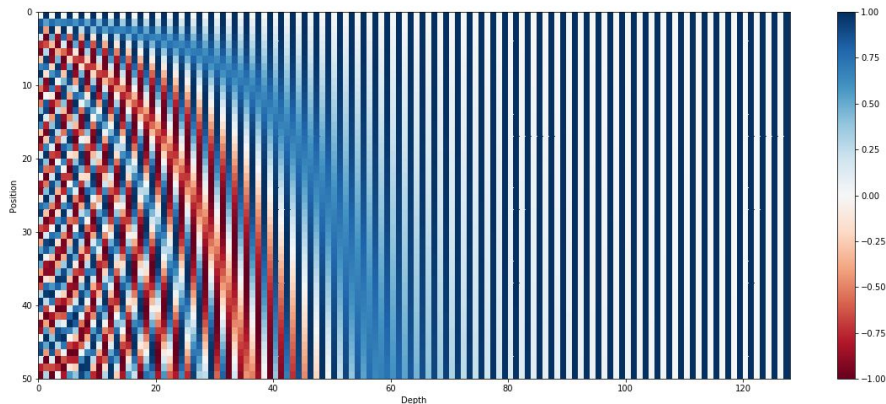
i: vector dimension

t: position

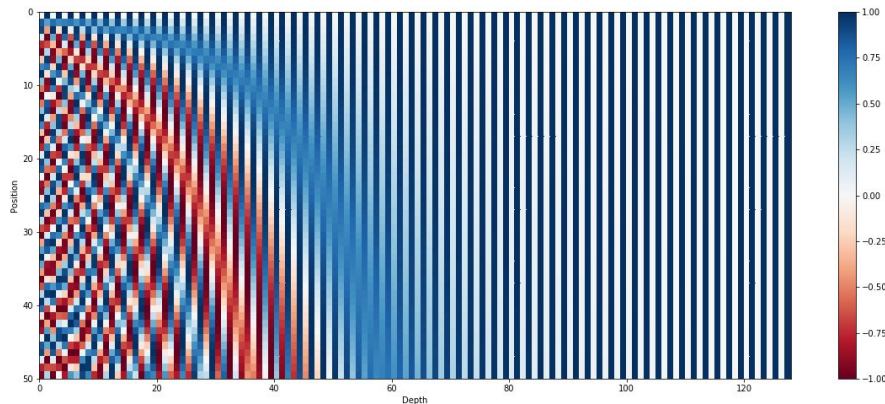
For example, in the original paper:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



Positional Encoding



0:	0	0	0	0	8:	1	0	0	0
1:	0	0	0	1	9:	1	0	0	1
2:	0	0	1	0	10:	1	0	1	0
3:	0	0	1	1	11:	1	0	1	1
4:	0	1	0	0	12:	1	1	0	0
5:	0	1	0	1	13:	1	1	0	1
6:	0	1	1	0	14:	1	1	1	0
7:	0	1	1	1	15:	1	1	1	1

"We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset ϕ , $PE(t+\phi)$ can be represented as a linear function of $PE(t)$." - i.e. the transformation matrix does not depend on the actual position, only on relative position

$$M \cdot \begin{bmatrix} \sin(\omega_k \cdot t) \\ \cos(\omega_k \cdot t) \end{bmatrix} = \begin{bmatrix} \sin(\omega_k \cdot (t + \phi)) \\ \cos(\omega_k \cdot (t + \phi)) \end{bmatrix}$$

where M does not depend on t

Transformers

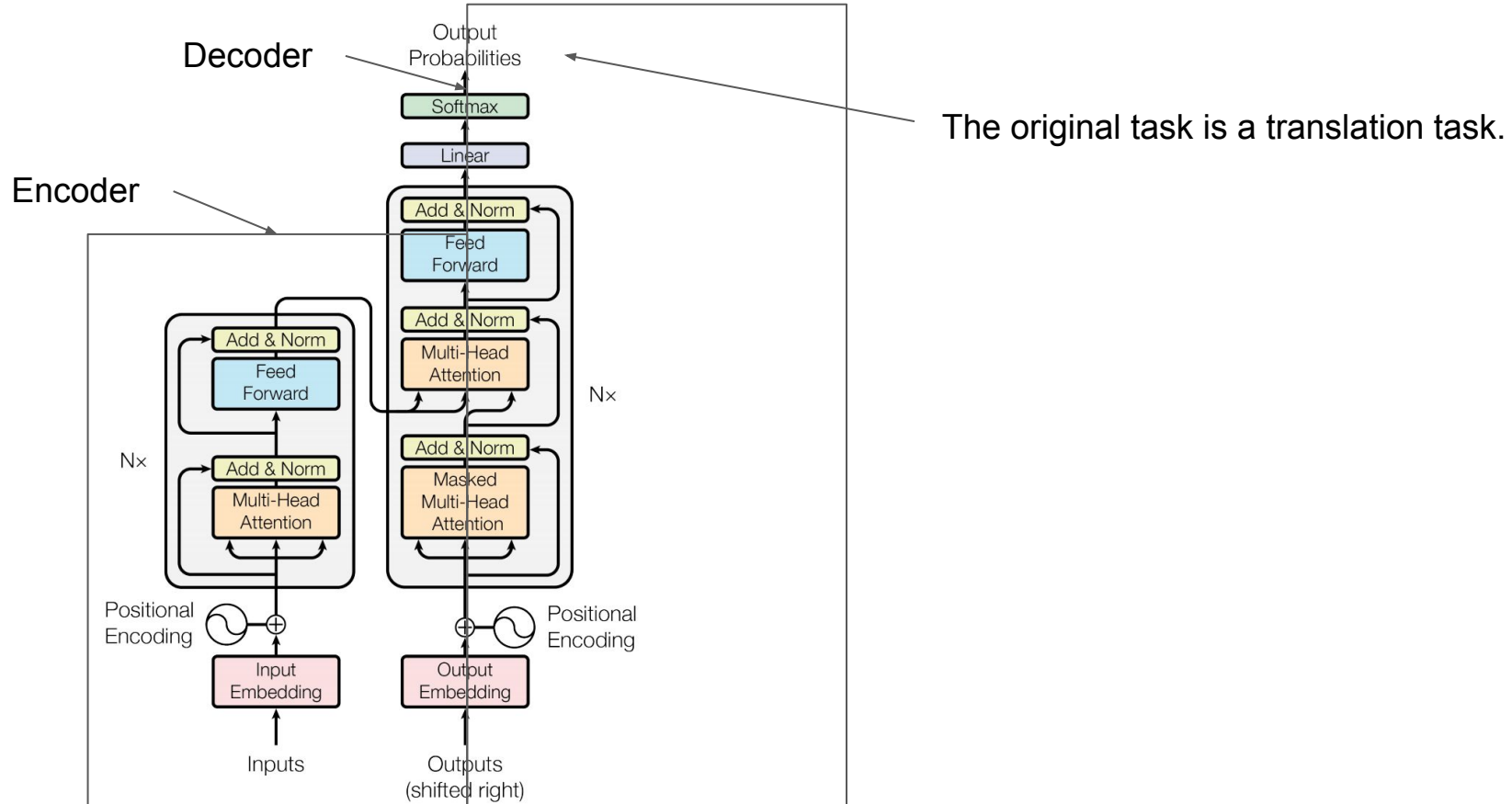
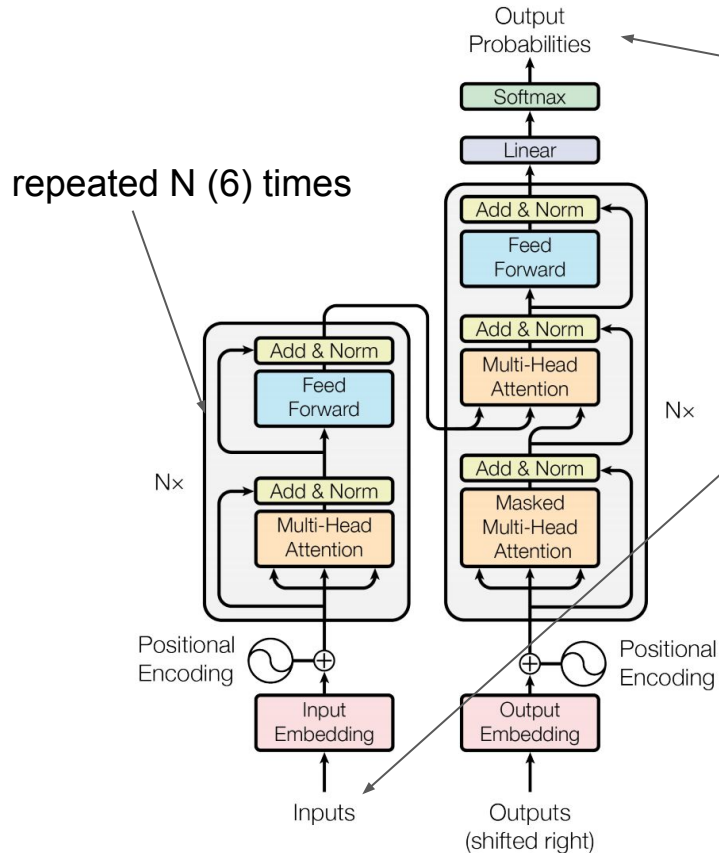


Figure 1: The Transformer - model architecture.

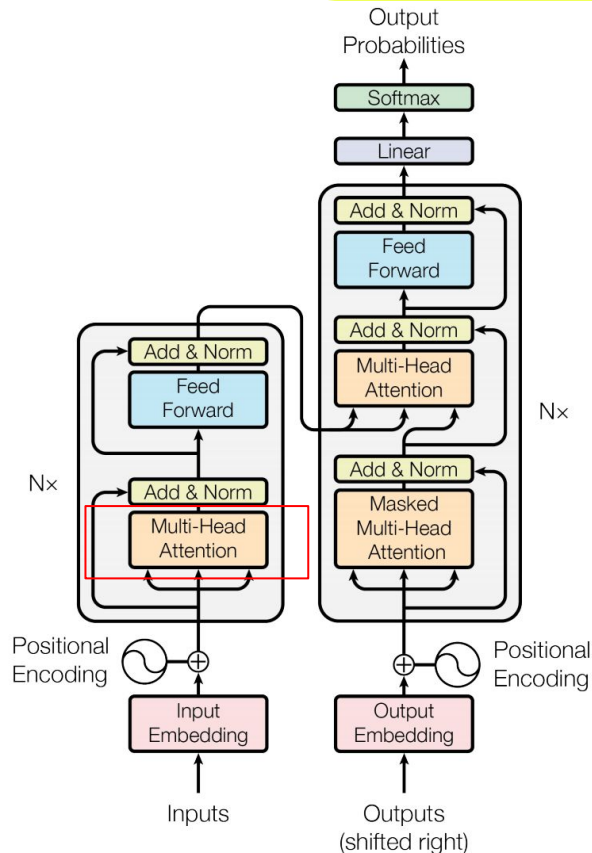
Transformers



- The original task is a translation task.
- The input takes all words in parallel
 - Padded up to a large maximum length, e.g. 512 or 1024.
- No recurrence - training can be parallelized

Figure 1: The Transformer - model architecture.

Transformers



- The original task is a translation task.
- The input takes all words in parallel
 - Padded up to a large maximum length, e.g. 512 or 1024.
- No recurrence - training can be parallelized
- Let's break down some of these

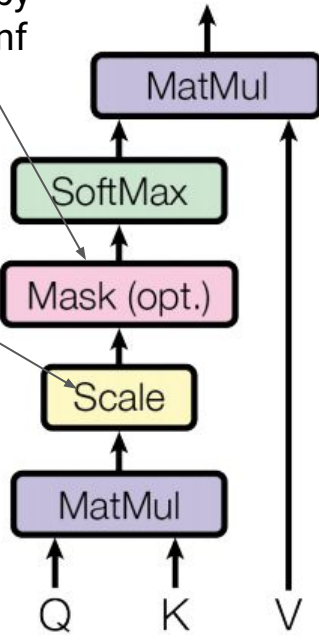
Figure 1: The Transformer - model architecture.

Multi-headed Attention

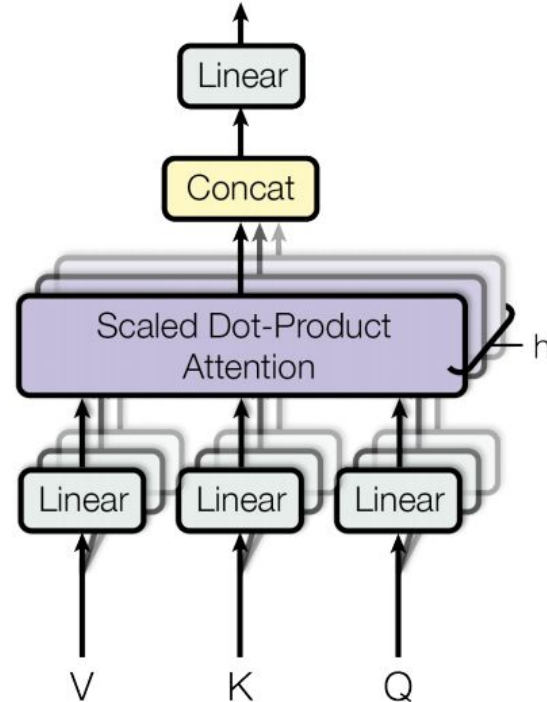
Scaled Dot-Product Attention

to mask padding by
setting value to $-\infty$
inside softmax

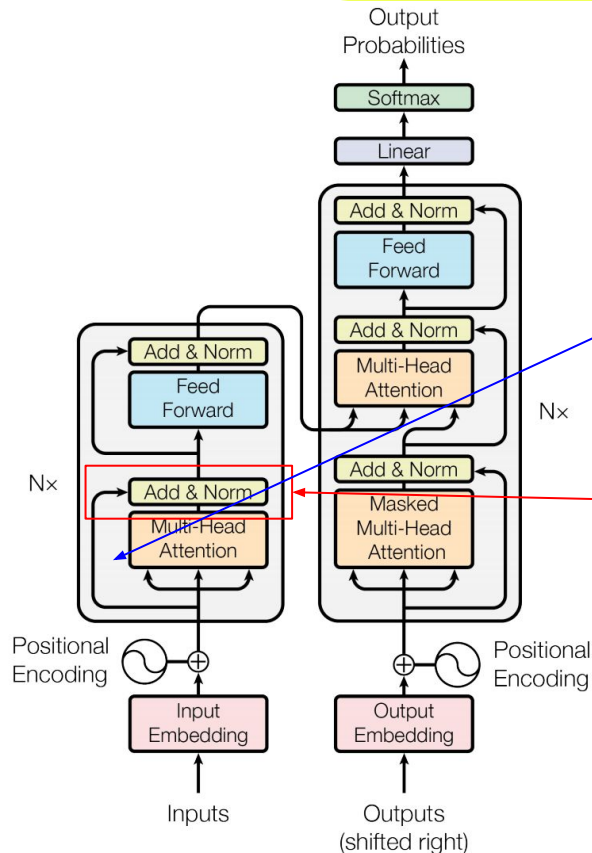
simple division
by $\sqrt{d_k}$,
where d_k is
dimension of
keys/queries



Multi-Head Attention



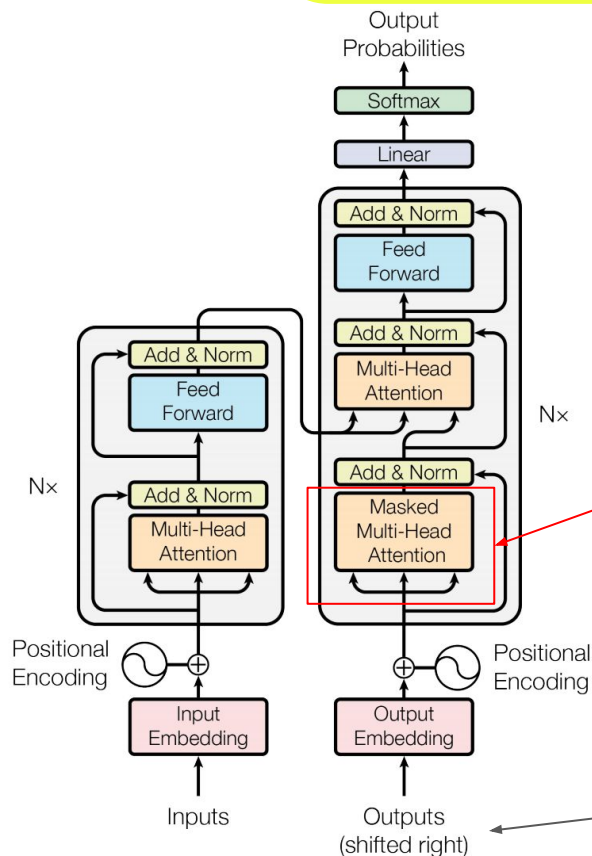
Transformers



- The skip-connection helps with Vanishing gradients (similar to residual connections for ResNet convolutional networks)
- After normalization, outputs sum up to 0 and have a standard deviation of 1. This helps with training, similar to batch normalization.

Figure 1: The Transformer - model architecture.

Transformers

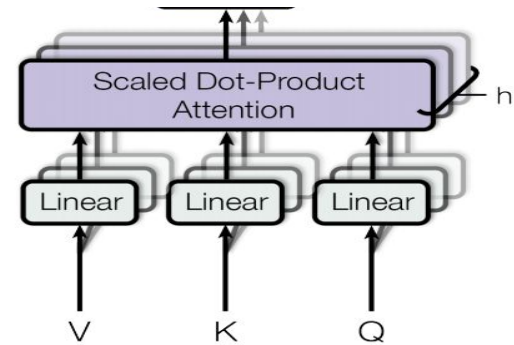
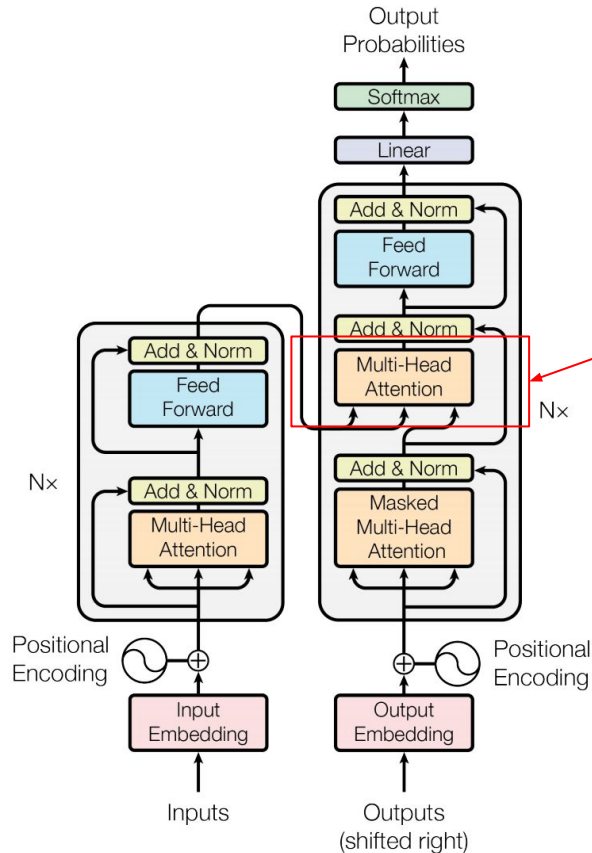


- Transformers process all inputs in parallel, meaning the answer is already in the training sequence.
- To prevent the model from "cheating" by looking at the next word, we apply a mask.
- Adding $-\text{Inf}$ to the upper triangle of the attention scores means that after passing through the Softmax function, those weights become exactly 0.
- The model can only attend to the current token and the tokens that came before it.

shifted right to insert the <START> token.

Figure 1: The Transformer - model architecture.

Transformers



Query from decoder using the already translated text, keys and values from encoder to look up the correct words for translating the next word.

Figure 1: The Transformer - model architecture.

Results

- **The Proving Ground:** The model was tested on the standard WMT 2014 Machine Translation datasets (4.5M pairs for EN-DE, 36M for EN-FR).
- **The Results:** The "Transformer (Big)" model achieved a BLEU score of 28.4 on EN-DE, establishing a new state-of-the-art in 2017.
- **Beating Ensembles:** This single model did not just beat other single models; it outperformed the best *ensembles* of previous architectures (like ConvS2S and GNMT) by over 2.0 BLEU points.

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Computational Complexity

- **Complexity per Layer:** Self-Attention requires $O(N^2d)$ operations, compared to Recurrent's $O(Nd^2)$. Because sequence length (N) is typically much smaller than the representation dimension (d) in standard text tasks, attention is fundamentally cheaper per layer.
- **Sequential Operations:** Recurrent models require $O(N)$ sequential steps. Self-Attention requires exactly $O(1)$, unlocking massive parallelization.
- **Maximum Path Length:** How far must information travel to connect the first and last word? Recurrent models take $O(N)$ steps, leading to vanishing gradients. Self-Attention connects any two tokens in $O(1)$ steps, flawlessly preserving long-range dependencies.

Computational Complexity

- **The Real Revolution is Cost:** The Transformer wasn't just better at translating; it was remarkably efficient to train.
- **Previous SOTA (GNMT + RL):** Took 3 weeks on 8 K80 GPUs, demanding a massive 1.0×10^{20} FLOPs.
- **Transformer (Big):** Trained in just 3.5 days on 8 P100 GPUs, using only 3.3×10^{18} FLOPs.
- By eliminating the recurrence bottleneck, the architecture saturates modern GPU compute cores to their maximum capacity, ensuring nearly every clock cycle is utilized for matrix multiplications rather than waiting on sequential states.

Ablation Studies

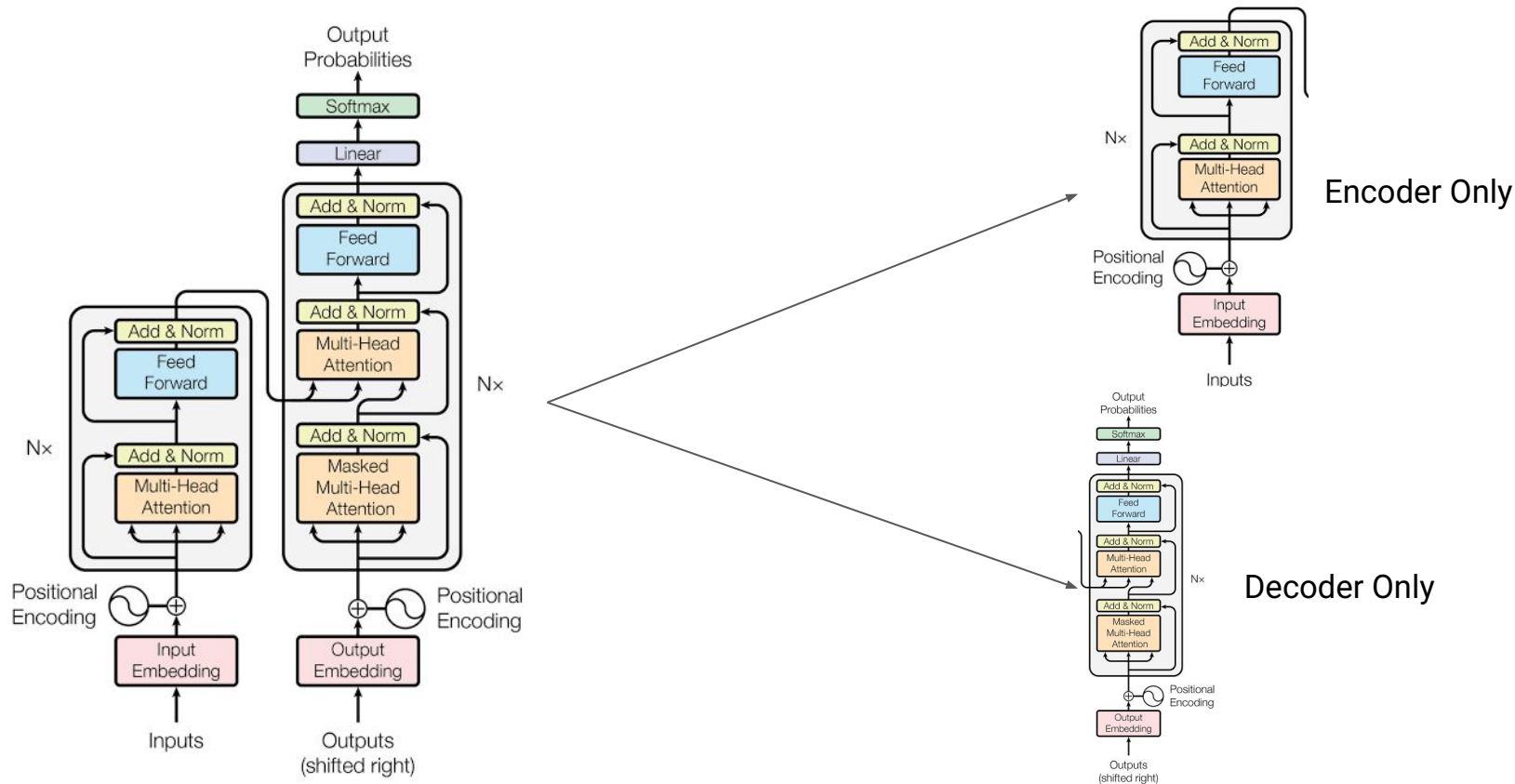
Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)					1	512				5.29	24.9	
					4	128				5.00	25.5	
					16	32				4.91	25.8	
					32	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
	256				32	32				5.75	24.5	28
	1024				128	128				4.66	26.0	168
			1024						5.12	25.4	53	
(D)									0.0	5.77	24.6	
									0.2	4.95	25.5	
									0.0	4.67	25.3	
									0.2	5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16				0.3	300K	4.33	26.4	213

What Actually Mattered?

- **Number of Heads (h):** More is not always better. The authors found a sweet spot at $h=8$ for their base model. Reducing to 1 head destroyed performance, but pushing to 16 or 32 heads actually caused a slight degradation.
- **Attention Key Size (d_k):** The scaling factor $1/\sqrt{d_k}$ is mathematically non-negotiable. Without it, larger key dimensions push the dot-products into regions where the softmax gradient approaches zero.
- **Positional Encodings:** The authors tested learned positional embeddings versus the fixed sinusoidal functions. They yielded nearly identical BLEU scores, proving the continuous, deterministic sine waves are fully sufficient without adding trainable parameters.
- **Dropout:** Standard regularization remains critical. A residual dropout rate of **10%** was vital to prevent overfitting the training data.

The Taxonomy of Transformers



The Taxonomy of Transformers

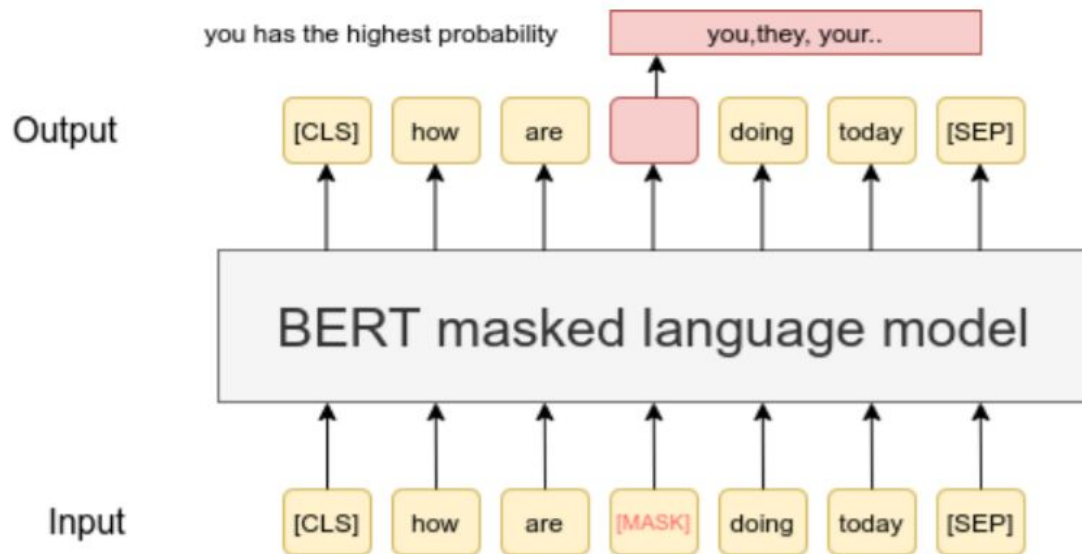
- Researchers quickly realized that you did not always need both halves of the original architecture.
- Different tasks require different types of contextual understanding.
- The architecture split into three distinct evolutionary branches based on how the text is masked and processed during training.

Encoder-Only

Masked Language Modeling (MLM)

- **The Objective:** Randomly mask a percentage of tokens in the input (usually around 15%) and force the network to predict the original words.
- **Bidirectional Context:** Because there is no causal mask in the encoder, the model looks at the entire sentence at once (both left and right of the target).
- **Best For:** Natural Language Understanding (NLU), sentiment analysis, text classification, and extracting deep contextual representations.
- **Famous Pre-cursor:** BERT (Bidirectional Encoder Representations from Transformers).

Encoder-Only

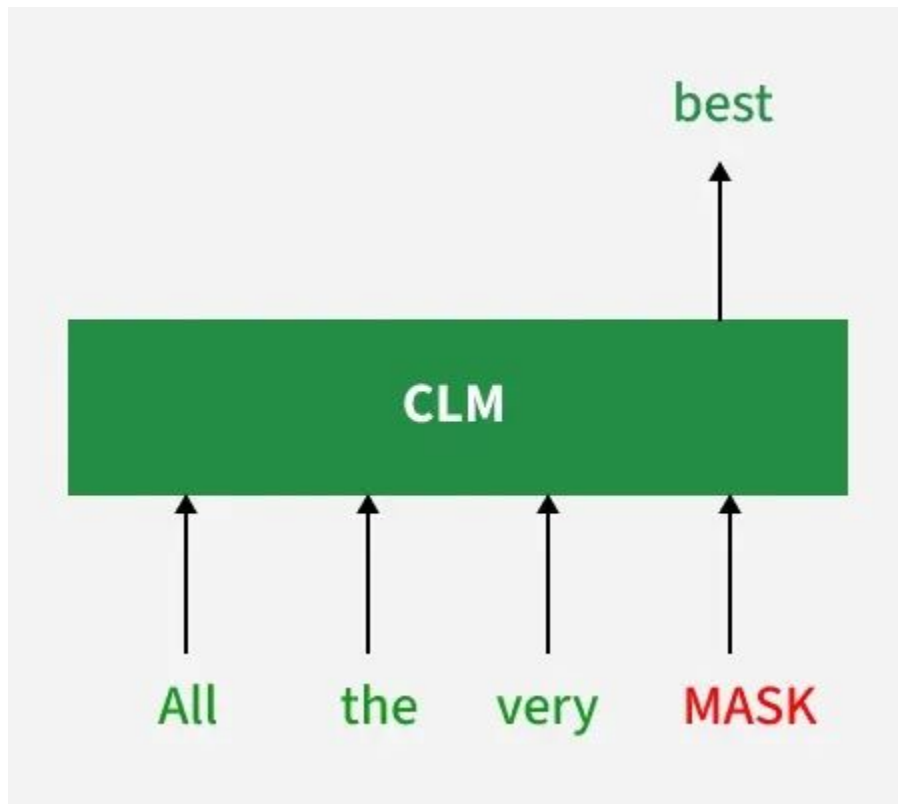


Decoder-Only

Causal Language Modeling (CLM)

- **The Objective:** Given a sequence of words, predict the very next token in the sequence.
- **Unidirectional Context:** The model is strictly constrained by the causal mask. It can only see the past, never the future.
- **Best For:** Natural Language Generation (NLG), writing code, creative writing, and open-ended dialogue.
- **Famous Pre-cursor:** The GPT (Generative Pre-trained Transformer) family.

Decoder-Only



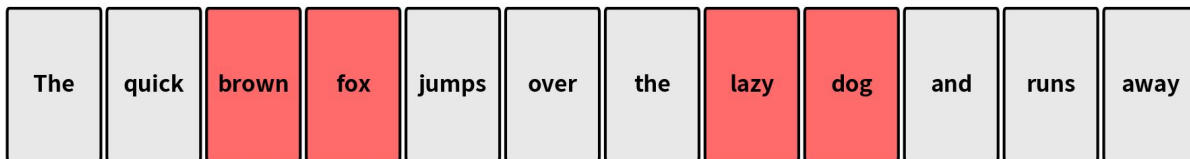
Encoder-Decoder

Span Corruption

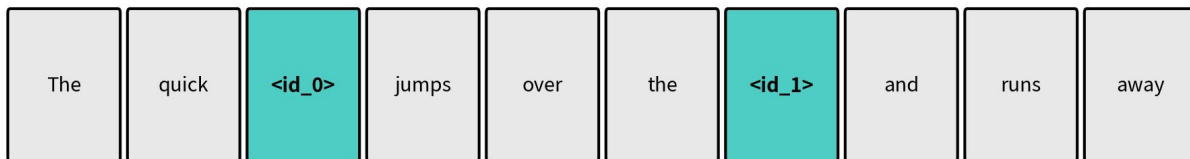
- **The Objective:** Keep the original architecture but corrupt continuous "spans" of text rather than isolated words. The decoder must reconstruct the missing spans.
- **The Hybrid Approach:** The encoder gets bidirectional context to understand the corrupted input, and the decoder autoregressively generates the missing pieces.
- **Best For:** Sequence-to-Sequence tasks like summarization, translation, and arbitrary text-to-text transformations.
- **Famous Pre-cursor:** T5 (Text-to-Text Transfer Transformer) and BART.

Encoder-Decoder

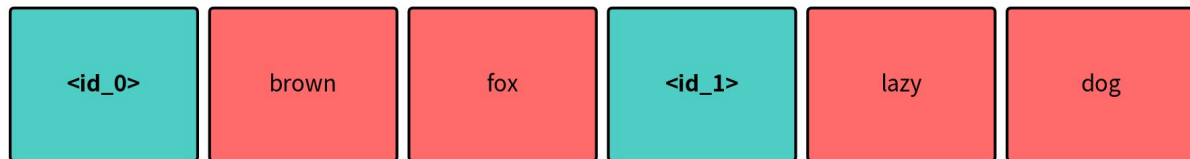
Original Sequence (corrupted spans highlighted in red)



Corrupted Input (sentinels in teal)



Target Sequence (sentinels delimit reconstructed spans)



Vision Transformers

- We know CNNs (ResNets, YOLO) dominate Computer Vision. But Transformers are incredibly powerful. Can we just use Self-Attention on an image?
- **The Problem:** An image is just a sequence of pixels.
- A standard 256x256 image contains **65,536 pixels**.
- Because Self-Attention complexity is $O(N^2)$, every pixel must be compared to every other pixel. That is a 65,536x65,536 attention matrix—over **4 billion operations** for a single layer! It would melt the GPU.

Vision Transformers

- In 2020, Google researchers realized they didn't need to treat every *pixel* as a token.
- **The Hack:** Split the image into a grid of non-overlapping patches (usually 16x16 pixels).
- Instead of 65,536 pixel tokens, a 256x256 image yields exactly **256 patch tokens**.
- We now treat each patch exactly like a "Word" in a sentence!

Vision Transformers

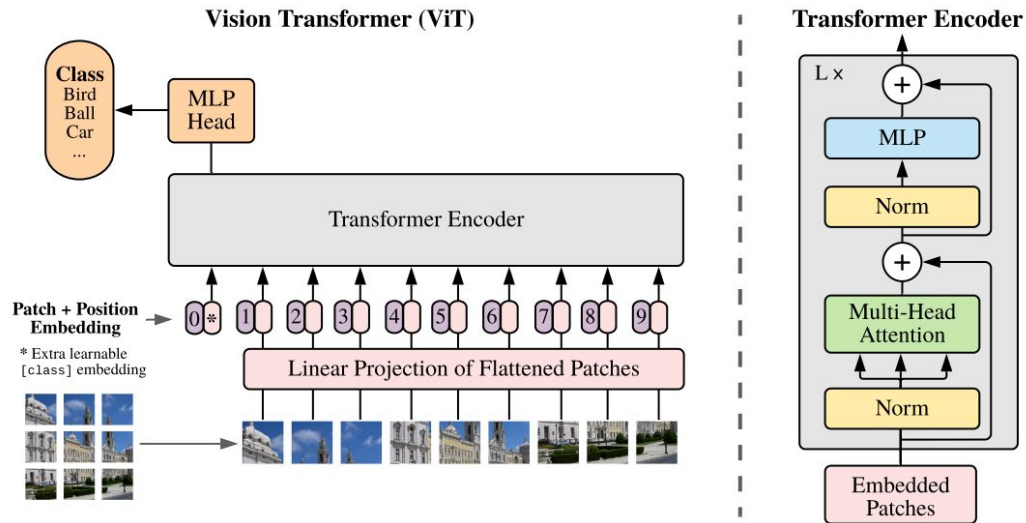


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

Patch Embeddings & Positional Encoding

- How do we turn a 2D image patch into a 1D word vector?
- **Step 1 (Flattening):** A 16x16 patch with 3 color channels (RGB) contains $16 \times 16 \times 3 = 768$ values. We flatten this into a single 1D vector of size 768.
- **Step 2 (Linear Projection):** We pass it through a standard Linear layer to create our final "Patch Embedding" (the equivalent of GloVe or Word2Vec for images).
- **Step 3 (Position):** Because the Transformer has no concept of 2D space, we add a learned 1D Positional Encoding to each patch so the network knows where the patch came from.

Vision Transformers

- We prepend one extra, blank, learnable token to the start of the sequence called the **[CLS] (Classification) Token**.
- We pass all 257 tokens into the **exact same Transformer Encoder** we just learned about (Multi-Head Attention + FFN + LayerNorm).
- The patches pay attention to each other to figure out the context of the image.
- At the very end, we grab the final state of the **[CLS]** token and pass it to a Multi-Layer Perceptron (MLP) to output our class probabilities (e.g., "Dog" or "Cat").

CNNs vs ViT

- Unlike CNNs, which build up feature maps step-by-step, ViT attention heads have a global receptive field from the very first layer.
- During training, the model naturally learns to cluster its attention around the most semantically meaningful parts of the image.
- The [CLS] token effectively learns to "query" all the background patches and subject patches, dynamically deciding which pixels actually matter for classification.

Attention Maps

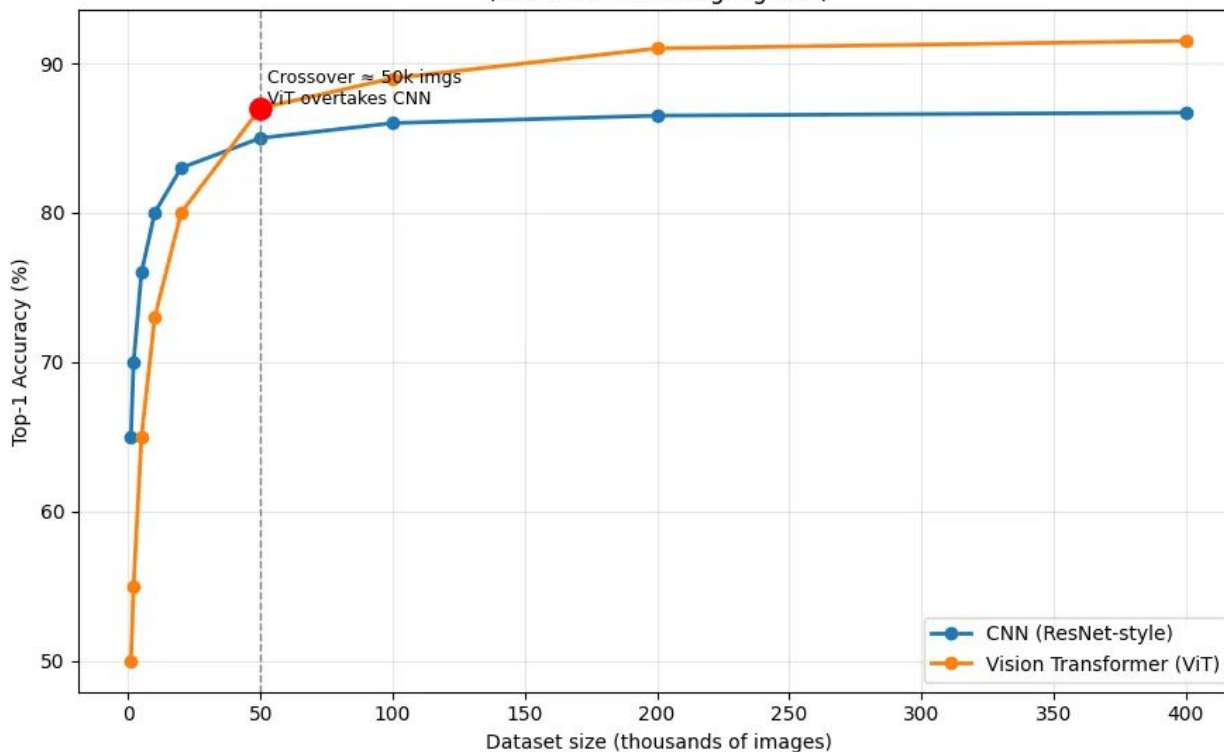


CNNs vs ViT

- **Inductive Bias:** CNNs inherently assume that pixels close to each other are related (translation invariance and locality).
- **Starting from Scratch:** ViTs have no structural understanding of 2D space. The 1D positional encodings must *learn* the 2D relationships of the patches through brute force.
- **The Data Hungry Reality:** Because it lacks inductive bias, a ViT performs worse than a ResNet on smaller datasets (like the 1.2 million images in ImageNet).
- **The Scaling Law:** When trained on massive, internet-scale datasets (like Google's private 300-million image JFT-300M dataset), the ViT crushes traditional CNNs.

CNNs vs ViT

ViT vs CNN Performance Across Dataset Sizes
(Crossover Point Highlighted)



Other Links

- [11. Attention Mechanisms and Transformers](#)
- [Stanford CS 224N Lecture 5: Transformers](#)
- [Attention Is All You Need](#)
- [An Image Is Worth 16x16 Words: Vision Transformers](#)